Deterministic Execution Control for Adaptive AI Systems

Technical Evidence Note | EKA execution-control program | August 2026

Evidence claim. The implemented control plane separates probabilistic/adaptive generation from deterministic execution authority. Candidate outputs are classified before execution into RESPOND, REFUSE, DEFER, or SILENCE; downstream actuation is permitted only when the deterministic layer admits continuation.

1. Purpose and scope

This note records the engineering behavior demonstrated by the EKA execution-control program. It is intentionally narrower than the upstream first-principles framework and narrower than the associated patent disclosure. The purpose is to make the execution-control mechanism inspectable without requiring agreement with the ontology from which the engineering question emerged.

2. Control architecture

The implementation treats model generation and execution authority as separate layers. The adaptive model may generate candidates; it does not decide whether a candidate is permitted to execute.

Adaptive model candidate generation	**Constraint interface** deterministic admissibility evaluation	**Outcome state** RESPOND / REFUSE / DEFER / SILENCE	**Execution interlock** actuation boundary	**External system** execution only if admitted

- Context-lock preserves the operative problem boundary before candidate evaluation.
- Admissibility evaluation is deterministic and external to the model's learned objective structure.
- Answer gating blocks coherent-but-unrealized outputs from reaching execution.
- Trajectory monitoring detects drift toward locally plausible but structurally inadmissible continuations.
- The execution interlock prevents actuation when the selected outcome is REFUSE, DEFER, or SILENCE.
- A working demonstration was implemented in App.py, with the same deterministic admissibility behavior exercised in online and fully offline modes.

3. Minimal execution outcomes

Outcome	Meaning	Execution consequence
RESPOND	Current continuation satisfies the deterministic gate.	Output may proceed.
REFUSE	Current candidate is incompatible with enforced constraints.	Execution is blocked.
DEFER	A required external or unresolved state prevents local completion.	No execution until the blocking condition changes.
SILENCE	No admissible local continuation is available.	No output is fabricated merely to maintain activity.

4. Recovery is re-entry, not override

State recovery is explicitly typed as a separate layer. It may modify an underspecified local state so that candidate generation and admissibility evaluation become meaningful again; it does not manufacture admissibility or bypass a refusal.

- Recovery operates on underspecified SILENCE states and certain REFUSE states under constraint retyping.
- Recovery does not operate on inconsistent SILENCE states.
- Recovery cannot create admissibility where none exists.
- Recovery cannot override invariance or bypass REFUSE conditions.
- Recovery returns the system to admissible evaluation; it is not continuation itself.

5. Demonstrated engineering properties

Property	Evidence represented in the build
Separation of authority	Model generation is probabilistic/adaptive; execution authority is deterministic.
Pre-execution gating	Constraint evaluation occurs before output or actuation rather than relying on post-hoc repair.
Bounded terminal behavior	REFUSE, DEFER, and SILENCE are valid outcomes rather than error states requiring forced continuation.
Local enforcement	Admissibility enforcement occurs locally within the deterministic supervisory layer.
Offline reproducibility	The same deterministic supervisory behavior was exercised in online and fully offline / air-gapped operation.
Model-weight independence	Execution control is enforced outside model weights.

6. Finite behavioral verification and freeze

Canonical completed engineering record: the finite behavioral target contains 13 criteria; 13 were demonstrated, 0 partially demonstrated, and 0 not yet demonstrated. The final verification state is INDEPENDENTLY_VERIFIED and the checkpoint state is FROZEN_AND_INDEPENDENTLY_VERIFIED.

- all_required_behavioral_demonstrations_independently_verified = true
- core_build_behaviorally_complete = true
- core_build_complete = true
- remaining_dependency_count = 0
- finite_stopping_condition_satisfied = true
- further_core_build_stage_licensed = false
- next_admissible_stage = STOP_CORE_BUILD_FINITE_TARGET_SATISFIED

Interpretation

The freeze establishes completion of the defined finite behavioral target. It does not establish universal AI safety, general correctness of the upstream ontology, or performance superiority over alternative control architectures.

7. Relationship to the allowed patent

The execution-control work is related to, but not identical with, the allowed U.S. patent application "Systems and Methods for State-Coherent Recursive Processing." The patent disclosure describes recursive state-coherence control through invariant monitoring, continuation evaluation, bounded state updates, deterministic feedback, distributed coordination, fail-safe behavior, historical-state continuity, gain modulation, and substrate adaptation.

The present note concerns the downstream execution-control layer: whether an adaptive system is permitted to continue or actuate under current constraints. The patent architecture addresses state-coherent recursive processing more broadly.

8. Evidence boundary

- Completed internal engineering record: deterministic supervisory control behavior, four-outcome gating, recovery typing, finite behavioral verification, and frozen core-build state.
- Implementation record: a working demonstration was implemented in App.py.
- Externally examined evidence: the related U.S. patent application progressed through substantive examination to allowance.
- Active work: Lean formalization of upstream representation/admissibility distinctions remains in progress.
- Unproven generalization: applicability to superintelligence-scale systems remains a research question, not a completed result.

9. Public inspection boundary

This note is intended to stand on its own as the public engineering summary. Only evidence that a reviewer can directly inspect from the public repository is treated here as a public citation surface.

Accordingly, the note should be read as a concise engineering record, not as a substitute for public source-code inspection. Where public implementation artifacts are not yet linked, the claims are explicitly bounded to the recorded build state.

- Publicly inspectable: the curated USPTO allowance extract showing that claims 1-17 were allowed and containing the examiner's reasons for allowance.
- Publicly inspectable: this technical evidence note, which summarizes the implemented control behavior, recovery boundary, recorded finite behavioral verification result, and scope limits.
- Not represented as independently inspectable public evidence here: unpublished build records, source code, private source documents, or upstream research papers that are not yet exposed through the repository.
- If additional implementation artifacts or source code are later made public, they should be linked separately rather than implied by inaccessible references.

10. Inspection path

Fastest technical reading order: control architecture -> outcome semantics -> recovery boundary -> finite verification -> patent relationship -> upstream formalization.